Linux Command Line

The Shell, Files, Permissions and SSH

Quan Nguyen

September 2026

Table of Contents

1. What Is the Shell?
2. Navigation
3. Manipulating Files and Directories
4. Pipelines
5. Permissions
6. SSH: Secure Connection and File Transfer

The Command Line = The Shell

- The **shell** is a program that takes keyboard commands and passes them to the OS to execute.
- Almost all Linux distributions supply `bash` (Bourne Again SHell), an enhanced replacement for `sh`.
- A **terminal emulator** (e.g., GNOME Terminal, konsole) is the GUI window used to access the shell.

The Shell Prompt

```
me@linuxbox:~$
```

- Format: username, machine name, current working directory, then $.
- If the prompt ends in # instead of $, the session has **superuser** privileges.
- Unrecognized input returns: `bash: command: command not found`.

Command History & Cursor Movement

- Up/Down arrows scroll through **command history** (bash remembers last 1000 commands by default).
- Left/Right arrows move the cursor within the current command line.
- History file: `~/.bash_history`.
- Search history incrementally: `Ctrl-r` then type search text.

Ending a Session

- Close the terminal emulator window, or
- Type `exit`, or
- Press `Ctrl-d`.

Filesystem Tree Basics

- Linux organizes files in a **hierarchical directory (tree) structure**.
- The first directory is the **root directory** (/).
- Unlike Windows, Linux has a **single filesystem tree** regardless of how many storage devices are attached — devices are **mounted** at various points on the tree.
- show all mounted devices: df -h

Device	What it is	Where it's mounted
Main SSD partition	Root filesystem	`/`
Second hard drive	Extra storage	`/home`
USB flash drive	Removable media	`/media/usb` or `/mnt/usbdrive`
DVD/CD drive	Optical media	`/media/cdrom`
Network share	Remote storage	`/mnt/nfs-share`

Core Navigation Commands

- `pwd` — Print name of current working directory
- `cd` — Change directory
- `ls` — List directory contents

```
me@linuxbox:~$ pwd
/home/me
me@linuxbox:~$ cd Desktop
me@linuxbox:~/Desktop$ ls
```

Absolute vs. Relative Pathnames

- **Absolute pathname**: starts with /, always begins at the root directory.
- **Relative pathname**: starts from the current working directory.
- Special notations: . (current directory), .. (parent directory), ~ (home directory).

```
cd .. # move up one level
cd ../.. # move up two levels
cd ~ # go to home directory
cd - # go to previous directory
```

Core File Manipulation Commands

- `cp` — Copy files and directories
- `mv` — Move / rename files and directories
- `mkdir` — Create directories
- `rm` — Remove files and directories
- `ln` — Create hard and symbolic links

Wildcards (Globbing)

Wildcard	Meaning
`*`	Matches any characters, including none
`?`	Matches any single character
`[characters]`	Matches any character in the set
`[!characters]`	Matches any character NOT in the set
`[[:class:]]`	Matches a character class (alpha, digit, ...)

class: alnum, alpha, digit, lower, upper

```
*.txt # any file ending in .txt
Data??? # Data + exactly 3 characters
[abc]* # begins with a, b, or c
BACKUP.[0-9][0-9][0-9] # Any file beginning with BACKUP. followed
     by exactly three numerals
[[:upper:]]* # Any file beginning with an uppercase letter
```

Dot Files

- Files/directories beginning with `.` are **hidden**.
- Hidden from normal `ls`, but shown with `ls -a` or `ls -A`.
- Wildcard `.*` also matches `.` and `..` — use `.[!.]*` to exclude them.

mkdir — Create Directories

```
mkdir directory...
mkdir dir1 # creates one directory
mkdir dir1 dir2 dir3 # creates three directories
```

cp — Copy Files and Directories

```
cp item1 item2 # copy item1 to item2
cp item... directory # copy multiple items into directory
cp dir1/* dir2 # copy all the files in dir1 into dir2
cp -r dir1 dir2 # recursive copy (required for directories)
```

Common options: `-a` (archive, preserves attributes), `-i` (interactive/confirm overwrite), `-r` (recursive), `-u` (update only newer files), `-v` (verbose)

mv — Move and Rename Files

```
mv item1 item2 # rename or move+overwrite
mv item... directory # move multiple items into directory
```

- If destination directory exists, source directory is moved *into* it.
- If destination does not exist, source is *renamed* to it.
- Shares `-i`, `-u`, `-v` options with `cp`.

rm — Remove Files and Directories

```
rm item...
rm -r dir1 # recursive delete (required for directories)
rm -rf dir1 # force, ignore nonexistent files, no prompts
```

Warning: No Undelete!

Linux has no "undelete" command. Always test wildcard patterns with `ls` before running `rm` on them. A stray space (e.g., `rm * .html`) can delete everything!

ln — Hard Links

```
ln file link
```

- Every file starts with one hard link (its name).
- Hard links share the same **inode** — verify with `ls -li`.
- Limitations: cannot cross filesystems; cannot link to directories.

ln -s — Symbolic Links

```
ln -s item link
```

- A special file containing a text pointer to the target.
- Can span filesystems and reference directories.
- Shown with a leading `l` in `ls -l` output: `lrwxrwxrwx ... link -> target`
- If the target is deleted, the link becomes **broken** (often shown in red by `ls`).

Pipelines and Filters

- | connects the output of one command to the input of another.
- Common filters: `sort`, `uniq`, `wc`, `grep`, `head`, `tail`, `tee`

```
ls /bin | sort | uniq | wc -l
ls /bin | grep zip
tail -f /var/log/messages
ls /bin | tee list.txt
```

Multi-User Model

- Linux is a **multi-user** system: multiple people can use the computer (locally or via SSH).
- Every file/directory has an **owner** and a **group owner**.
- Access rights are granted separately to: **owner**, **group**, and **others** (world).
- `id` — displays your uid, gid, and group memberships.

Reading a Long Listing

```
> foo.txt
ls -l foo.txt
-rw-rw-r-- 1 me me 0 2016-03-06 14:52 foo.txt
```

- First character: file type (`-` regular, `d` directory, `l` symlink, `c`/`b` device files).
- Remaining 9 characters: permissions for **owner**, **group**, **other** — each a group of `rwx`.

Permission Attributes: r, w, x

Attr	On Files	On Directories
`r`	Open/read file	List directory contents
`w`	Write/truncate file	Create/delete/rename files (needs x)
`x`	Execute as program	Enter directory (`cd`), access metadata

chmod: Octal Notation

Octal	Binary	Mode
0	000	—
4	100	r–
5	101	r-x
6	110	rw-
7	111	rwx

```
chmod 600 foo.txt # rw------- (owner only)
chmod 755 script.sh # rwxr-xr-x
```

chmod: Symbolic Notation

- Who: u (user/owner), g (group), o (others), a (all)
- Operation: + (add), - (remove), = (set exactly)

```
chmod u+x script.sh # add execute for owner
chmod u-x script.sh # remove execute from owner
chmod o-rw file.txt # remove read/write from others
chmod go+rx,u+x file # multiple specs, comma-separated
```

umask: Default Permissions

- Controls default permissions for newly created files.
- Expressed as an octal **mask** of bits to remove.

```
umask # view current mask (e.g., 0002)
umask 0022 # set new mask
```

```
Original file mode: rw- rw- rw-
Mask: 000 000 010 (0002)
Result: rw- rw- r--
```

```
Original file mode: rw- rw- rw-
Mask: 000 010 010 (0022)
Result: rw- r-- r--
```

Changing Identities: su and sudo

- `su -` — start a login shell as another user (default: root); requires **that user's** password.
- `sudo command` — execute a single command with elevated privileges; requires **your own** password.
- sudo does not start a new shell/environment
- Modern distros (e.g., Ubuntu) disable direct root login and rely on `sudo`.

```
su [-[l]] [user]
su -c "ls -l /root"
sudo systemctl restart nginx
sudo -l # list allowed sudo privileges
```

chown and chgrp

```
chown owner:group file...
chown bob file.txt # change owner to bob
chown bob:users file.txt # change owner and group
chown :admins file.txt # change group only
chgrp admins file.txt # older, group-only equivalent
```

- Requires superuser privileges.

Practical Example: Shared Directory

```
sudo groupadd music
sudo usermod -a -G music janet
sudo usermod -a -G music tony
sudo mkdir /usr/local/share/Music
sudo chown :music /usr/local/share/Music
sudo chmod g+s /usr/local/share/Music
```

Creates a shared group `music`, adds two users, and configures a directory so new files inherit the group.

What Is SSH?

SSH (Secure Shell) is a protocol/command-line tool that lets you securely log in to and control a remote server.

- **Authenticates:** verifies the remote host's identity before any data is exchanged.
- **Encrypts:** the entire session between the local machine and the server is encrypted end-to-end.
- Servers (VPS) typically **have no graphical interface** — SSH is the primary way to operate them from the command line.

Basic Syntax

```
$ ssh user@<vps-address>
```

Authenticating with an SSH Key Pair

Instead of a password, SSH commonly uses an **asymmetric key pair**:

Private key

- Stored on the local machine.
- Keep absolutely secret, never share it.

Public key

- Stored on the server.
- Used to authenticate the remote user.

Connection process:

1. The local machine uses the private key to create a connection request.
2. The server uses the public key to verify the request.
3. If valid, an encrypted connection is successfully established.

Generating Keys and Connecting

`ssh-keygen` — Generate a new key pair

Creates a public/private key pair on the local machine.

`ssh` — Connect to the server

Logs in to the VPS using the generated private key.

```
user@linux:~$ ssh-keygen
user@linux:~$ ssh user@<vps-address>
```

Copying Files: `scp` and `rsync`

scp — Secure Copy

- Copies **a single file** over an encrypted connection.
- Best for quick, simple transfers.

rsync — Synchronization

- Synchronizes **entire directories**.
- Best for transferring many files or repeated updates.

```
$ scp file.txt user@<vps-address>:/path/
$ rsync -av folder/ user@<vps-address>:/path/
```

`sftp` is an interactive alternative — a secure replacement for traditional `ftp`.

Checking Network Connectivity

Before and while working with SSH, the following commands help diagnose connectivity to the server:

- **ping** — tests reachability, latency, and packet loss.
- **traceroute** — shows every hop a packet takes to reach the destination host.

Note

If `ssh` fails to connect, check the network with `ping` first before suspecting key configuration or firewall issues.